

Yggdrasil 服务端技术规范

概述

本文旨在为实现 Yggdrasil 服务端提供非官方的技术规范。

本规范中所描述的服务端行为不一定与 Mojang 服务端的相同。这客观上是因为 Mojang 的服务端是闭源的，我们只能推测其内部逻辑，而所作出的推测难免会与实际存在出入。但事实上只要客户端能够正确理解并处理服务端的响应，那么其行为是否与 Mojang 服务端的相同，也就无关紧要了。

基本约定

字符编码

本文中字符编码一律使用 UTF-8。

请求与响应格式

若无特殊说明，请求与响应均为 JSON 格式（如果有 body），Content-Type 均为 `application/json; charset=utf-8`。

所有 API 都应该使用 HTTPS 协议。

错误信息格式

```
{
  "error": "错误的简要描述（机器可读）",
  "errorMessage": "错误的详细信息（人类可读）",
  "cause": "该错误的原因（可选）"
}
```

当遇到本文中已说明的异常情况时，返回的错误信息应符合对应的要求。

下表列举了常见异常情况下的错误信息。除特殊说明外，`cause` 一般不包含。

非标准指由于无法使 Mojang 的 Yggdrasil 服务器触发对应异常，而只能推测该种情况下的错误信息。

未定义指该项并没有明确要求。

异常情况	HTTP 状态 码	Error	Error Message
一般 HTTP 异常 (非业务异常， 如 <i>Not Found</i> 、 <i>Method Not Allowed</i>)	未定 义	该 HTTP 状态对应的 <i>Reason Phrase</i> (于 HTTP/1.1 中定义)	未定义
令牌无效	403	ForbiddenOperationException	Invalid token.
密码错误，或短 时间内多次登录 失败而被暂时禁 止登录	403	ForbiddenOperationException	Invalid credentials. Invalid username or password.
试图向一个已经 绑定了角色的令 牌指定其要绑定 的角色	400	IllegalArgumentException	Access token already has a profile assigned.
试图向一个令牌 绑定不属于其对 应用户的角色 (非标准)	403	ForbiddenOperationException	未定义
试图使用一个错 误的角色加入服 务器	403	ForbiddenOperationException	Invalid token.

数据格式

我们约定以下数据格式

- **无符号 UUID:** 指去掉所有 - 字符后的 UUID 字符串

模型

用户

一个系统中可以存在若干个用户，用户具有以下属性：

- ID
- 邮箱
- 密码

其中 ID 为一个无符号 UUID。邮箱可以变更，但需要保证唯一。

用户信息的序列化

用户信息序列化后符合以下格式：

```
{
  "id":"用户的 ID",
  "properties":[ // 用户的属性（数组，每一元素为一个属性）
    { // 一项属性
      "name":"属性的名称",
      "value":"属性的值",
    }
    // ,...（可以有更多）
  ]
}
```

用户属性中目前已知的项目如下：

名称	值
preferredLanguage	(可选) 用户的偏好语言，例如 en、zh_CN

角色 (Profile)

Mojang 当前不支持多角色，不保证多角色部分内容的正确性。

角色与账号为多对一关系。一个角色对应 Minecraft 中的一个实体玩家。角色具有以下属性：

- UUID
- 名称
- 材质模型，可选值有：default、slim
 - **default**：正常手臂宽度（4px）的皮肤

- **slim**: 细手臂 (3px) 的皮肤
- 材质
 - 类型为映射
 - key 可选值有: SKIN、CAPE
 - value 类型为 URL

UUID 和名称均为全局唯一, 但名称可变。应避免使用名称作为标识。

角色 UUID 的生成

若不考虑兼容性, 角色的 UUID 一般为随机生成 ([Version 4](#))。

但 Minecraft 仅使用 UUID 作为角色标识符, 不同 UUID 的角色即使名称相同也被认为是不同的。如果一个 Minecraft 服务器从其他登录系统 (正版验证、离线验证或其他) 迁移到本登录系统, 并且角色的 UUID 发生了变化, 则该角色的数据将丢失。为了避免这种情况, 必须保证对于同一个角色, 本系统生成的 UUID 与其在先前系统中的 UUID 是相同的。

兼容离线验证

若 Minecraft 服务器原先采用的是离线验证, 则角色 UUID 是角色名称的一元函数。如果 Yggdrasil 服务端使用此方法生成角色 UUID, 就可以实现与离线验证系统之间的双向兼容, 即可以在不丢失角色数据的情况下, 在离线验证系统和本登录系统之间切换。

从角色名称计算角色 UUID 的代码如下 (Java):

```
UUID.nameUUIDFromBytes(("OfflinePlayer:" +  
characterName).getBytes(StandardCharsets.UTF_8))
```

在其他语言中的实现:

- [PHP](#)

角色信息的序列化

角色信息序列化后符合以下格式:

```
{
  "id": "角色 UUID（无符号）",
  "name": "角色名称",
  "properties": [ // 角色的属性（数组，每一元素为一个属性）（仅在特定情况下需要包含）
    { // 一项属性
      "name": "属性的名称",
      "value": "属性的值",
      "signature": "属性值的数字签名（仅在特定情况下需要包含）"
    }
    // ,...（可以有更多）
  ]
}
```

角色属性（`properties`）及数字签名（`signature`）在无特殊说明的情况下不需要包含。

`signature` 是属性值的数字签名，使用 Base64 编码。签名算法为 SHA1withRSA，见 [PKCS #1](#)。关于签名密钥的详细介绍，见 [签名密钥对](#)。

角色属性中可以包含以下项目：

名称	值
textures	（可选）Base64 编码的 JSON 字符串，包含了角色的材质信息，详见 § textures 材质信息属性 。
uploadableTextures	（可选）该角色可以上传的材质类型，为 authlib-injector 自行规定的属性，详见 § uploadableTextures 可上传的材质类型

textures 材质信息属性

以下为材质信息的格式，将这段 JSON 进行 Base64 编码后，即为 `textures` 角色属性的值。

```
{
  "timestamp": 该属性值被生成时的时间戳（Java 时间戳格式，即自 1970-01-01 00:00:00 UTC 至今经过的毫秒数），
  "profileId": "角色 UUID（无符号）",
  "profileName": "角色名称",
  "textures": { // 角色的材质
    "材质类型（如 SKIN）": { // 若角色不具有该项材质，则不必包含
      "url": "材质的 URL",
      "metadata": { // 材质的元数据，若没有则不必包含
        "名称": "值"
        // ,...（可以有更多）
      }
    }
    // ,...（可以有更多）
  }
}
```

材质元数据中目前已知的项目有 `model`，其对应角色的材质模型，取值为 `default` 或 `slim`。

uploadableTextures 可上传的材质类型

注意：这一角色属性是由 authlib-injector 文档规定的，Mojang 返回的角色属性是不包含这一项的。Mojang 仅允许用户上传皮肤，不允许上传披风。

考虑到并非所有验证服务器都允许用户上传皮肤和披风，因此 authlib-injector 规定了 `uploadableTextures` 角色属性，其表示角色可以上传的材质类型。

该属性的值是一个逗号分隔的列表，包含了可以上传的材质类型。材质类型目前有 `skin` 和 `cape` 两种。

例如，`uploadableTextures` 属性的值若为 `skin`，则表示可以为该角色上传皮肤，但不能上传披风；值若为 `skin,cape`，则既可以上传皮肤，又可以上传披风。

如果不存在 `uploadableTextures` 属性，则不能为该角色上传任何类型的材质。

关于材质上传接口的介绍，请参考 [§材质上传](#)。

材质 URL 规范

Minecraft 将材质 hash 作为材质的标识。每当客户端下载一个材质后，便会将其缓存在本地，以后若需要相同 hash 的材质，则会直接使用缓存。而这个 hash 并不是由客户端计算的。Yggdrasil 服务端应先计算好材质 hash，将其作为材质 URL 的文件名，即从 URL 最后一个 `/`（不包括）开始一直到结尾的这一段子串。而客户端会直接将 URL 的文件名作为材质的 hash。

例如下面这个 URL，它所代表的材质的 hash 为

`e051c27e803ba15de78a1d1e83491411dfffb6d7fd2886da0a6c34a2161f7ca99`：

`https://yggdrasil.example.com/textures/e051c27e803ba15de78a1d1e83491411dfffb6d7fd2886da0a6c34a2161f7ca99`

安全警告：

- 材质 URL 响应头中的 `Content-Type` 必须为 `image/png`。若未指定，则存在 MIME Sniffing Attack 的风险。
-

材质 hash 的计算方法服务端可自行选择，建议使用 SHA-256 或者更加安全的 hash 算法。作为参考，Mojang 官方的方法是计算图像文件的 SHA-256。

安全警告：

- 若不对用户上传材质进行处理，则**可能导致远程代码执行**
- 在读取材质前，若不先检查图像大小，则**可导致拒绝服务攻击**

关于此安全缺陷的详细信息：[未经检查的用户上传材质可能导致远程代码执行 #10](#)

除了位图数据外，PNG 文件还可以存储其他数据。如果 Yggdrasil 服务端不对用户上传的材质进行检查，则攻击者可以在其中藏匿恶意代码，并通过 Yggdrasil 服务端分发到客户端。因此，Yggdrasil 服务端**必须**对用户上传的材质进行处理，除去其中任何与位图无关的数据。具体做法如下：

1. 读取该 PNG 文件中图像的大小，如果过大则应拒绝。
 - 即使是非常小的 PNG 文件也可以存储一幅足以消耗计算机所有内存的图像（即 PNG Bomb），因此切不可在检查图像大小前就将其完整读入。
2. 检查图像是否为合法的皮肤/披风材质。
 - 皮肤的宽高为 64x32 的整数倍或 64x64 的整数倍，披风的宽高为 64x32 的整数倍或 22x17 的整数倍。宽高为 22x17 整数倍的披风并非标准尺寸的披风，服务端需要用透明像素将其宽高补足至 64x32 的整数倍。
3. 将图像文件重新保存，去除无关的元数据。

实现提示：在 Java 中可使用 `ImageReader.getWidth()` 在不读入整个图像的情况下获取其尺寸。

令牌 (Token)

令牌与账号为多对一关系。令牌是一种登录凭证，具有时效性。令牌具有以下属性：

- accessToken
- clientToken
- 绑定的角色
- 颁发时间

其中 accessToken 和 clientToken 为任意字符串（可以是无符号 UUID 或 JWT）。accessToken 由服务端随机生成，clientToken 由客户端提供。

鉴于 accessToken 的随机性，它可以被作为主键。而 clientToken 不具有唯一性。

绑定的角色可以为空。它代表了能使用该令牌进行游戏的角色。

一个用户可以同时有多个令牌，但服务端也应该对令牌数量加以限制。当令牌数量超出限制（如 10 个）时，则应先吊销最旧的令牌，之后再颁发新的令牌。

令牌的状态

令牌有以下三种状态：

- **有效**
 - 处于该状态的令牌可以进行各项操作，如[进服验证](#)、[刷新](#)。
 - 新颁发的令牌（即通过[登录](#)、[刷新](#)颁发的令牌）处于该状态。
- **暂时失效**
 - 处于该状态的令牌除了进行[刷新操作](#)外，无权进行任何操作。
 - 当令牌绑定的角色改名后，令牌应被标记为**暂时失效**状态。
 - 这是为了让启动器刷新令牌，从而获取到新的角色名称。（见 [#40](#)）
 - *该状态并不是必须要实现的（详细介绍见下）。*
- **无效**
 - 处于该状态的令牌无权进行任何操作。
 - 令牌被吊销后处于该状态。这里的吊销包括[显式吊销](#)、[登出](#)、[刷新](#)后吊销原令牌、令牌过期。

令牌的状态只能由有效变为无效，或是由有效变为暂时失效再变为无效，这个过程是不可逆的。刷新操作仅颁发一个新的令牌，并不能使原令牌重新回到有效状态。

令牌应当有一个过期时限（如 15 天）。当自颁发起所经过的时间超过该时限，令牌过期。

关于暂时失效状态

Mojang 对暂时失效状态的实现是这样的：对启动器而言，若令牌处于暂时失效状态，则会刷新令牌，获得一个新的处于有效状态的令牌；对 Yggdrasil 服务端而言，仅最后颁发的令牌才是有效的，先前颁发的其它令牌都处于暂时失效状态。

Mojang 之所以这么做，可能是为了防止用户多地同时登录（仅使最后一个 session 有效）。事实上，即使服务端没有实现暂时失效状态，启动器的逻辑也是可以正常工作的。

当然，就算我们要实现暂时失效状态，也并不需要以 Mojang 的实现为范本。只需要启动器能够正确处理，任何实现都是可以的。下面给出一个不同于 Mojang 的实现例子：

取一个短于令牌过期时限的时间段作为有效和暂时失效的分界点。若自颁发起经过的时间在该时限内，则令牌有效；若超过该时限，但仍在过期时限内，则令牌暂时失效。

这种做法实现了这样的功能：玩家如果经常进行登录操作，除了第一次登录就不需要输入密码了。而当他长时间未登录时则需要重新输入密码。

Yggdrasil API

用户部分

登录

POST /authserver/authenticate

使用密码进行身份验证，并分配一个新的令牌。

请求格式：

```
{
  "username": "邮箱（或其他凭证，详见 §使用角色名称登录）",
  "password": "密码",
  "clientToken": "由客户端指定的令牌的 clientToken（可选）",
  "requestUser": true/false, // 是否在响应中包含用户信息，默认 false
  "agent": {
    "name": "Minecraft",
    "version": 1
  }
}
```

若请求中未包含 `clientToken`，服务端应该随机生成一个无符号 UUID 作为 `clientToken`。但需要注意 `clientToken` 可以为任何字符串，即请求中提供任何 `clientToken` 都是可以接受的，不一定要为无符号 UUID。

对于令牌要绑定的角色：若用户没有任何角色，则为空；若用户仅有一个角色，那么通常绑定到该角色；若用户有多个角色，通常为 `空`，以便客户端进行选择。也就是说如果绑定的角色为 `空`，则需要客户端进行角色选择。

响应格式：

```

{
  "accessToken": "令牌的 accessToken",
  "clientToken": "令牌的 clientToken",
  "availableProfiles": [ // 用户可用角色列表
    // ... 每一项为一个角色（格式见 $角色信息的序列化）
  ],
  "selectedProfile": {
    // ... 绑定的角色，若为空，则不需要包含（格式见 $角色信息的序列化）
  },
  "user": {
    // ... 用户信息（仅当请求中 requestUser 为 true 时包含，格式见 $用户信息的序列化）
  }
}

```

安全提示： 该 API 可以被用于密码暴力破解，应受到速率限制。限制应针对用户，而不是客户端 IP。

使用角色名称登录

除使用邮箱登录外，验证服务器还可以允许用户使用角色名称登录。要实现这一点，验证服务器需要进行以下工作：

- 将 API 元数据中的 `feature.non_email_login` 字段设置为 `true`。（见 [API 元数据获取\\$功能选项](#)）
- 接受在[登录接口](#)中使用角色名称作为 `username` 参数。

当用户使用角色名称登录时，验证服务器应**自动将令牌绑定到相应角色**，即上文响应中的 `selectedProfile` 应为用户登录时所用的角色。

这种情况下，如果用户拥有多个角色，那么他可以省去选择角色的操作。考虑到某些程序不支持多角色（例如 Geyser），还可以通过上述方法绕过角色选择。

刷新

POST /authserver/refresh

吊销原令牌，并颁发一个新的令牌。

请求格式：

```

{
  "accessToken": "令牌的 accessToken",
  "clientToken": "令牌的 clientToken（可选）",
  "requestUser": true/false, // 是否在响应中包含用户信息，默认 false
  "selectedProfile": {
    // ... 要选择的角色（可选，格式见 $角色信息的序列化）
  }
}

```

当指定 `clientToken` 时，服务端应检查 `accessToken` 和 `clientToken` 是否有效，否则只需要检查 `accessToken`。

颁发的新令牌的 `clientToken` 应与原令牌的相同。

如果请求中包含 `selectedProfile`，那么这就是一个选择角色的操作。此操作要求原令牌所绑定的角色为空，而新令牌则将绑定到 `selectedProfile` 所指定的角色上。如果不包含 `selectedProfile`，那么新令牌所绑定的角色和原令牌相同。

刷新操作在令牌暂时失效时依然可以执行。若请求失败，原令牌依然有效。

响应格式：

```
{
  "accessToken": "新令牌的 accessToken",
  "clientToken": "新令牌的 clientToken",
  "selectedProfile": {
    // ... 新令牌绑定的角色，若为空，则不需要包含（格式见 §角色信息的序列化）
  },
  "user": {
    // ... 用户信息（仅当请求中 requestUser 为 true 时包含，格式见 §用户信息的序列化）
  }
}
```

验证令牌

POST /authserver/validate

检验令牌是否有效。

请求格式：

```
{
  "accessToken": "令牌的 accessToken",
  "clientToken": "令牌的 clientToken（可选）"
}
```

当指定 `clientToken` 时，服务端应检查 `accessToken` 和 `clientToken` 是否有效，否则只需要检查 `accessToken`。

若令牌有效，服务端应返回 HTTP 状态 204 No Content，否则作为令牌无效的异常情况处理。

吊销令牌

POST /authserver/invalidate

吊销给定令牌。

请求格式：

```
{
  "accessToken":"令牌的 accessToken",
  "clientToken":"令牌的 clientToken（可选）"
}
```

服务端只需要检查 `accessToken`，即无论 `clientToken` 为何值都不会造成影响。

无论操作是否成功，服务端应返回 HTTP 状态 `204 No Content`。

登出

POST `/authserver/signout`

吊销用户的所有令牌。

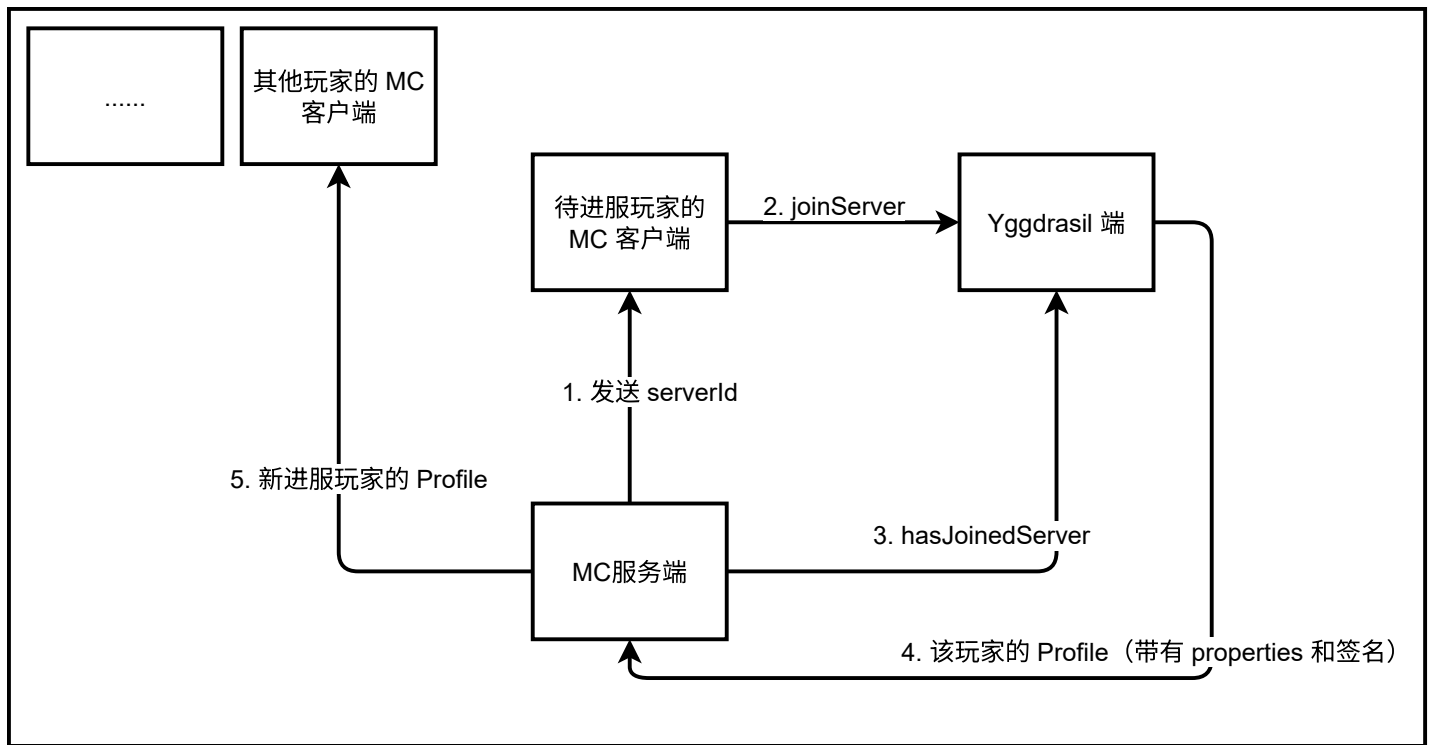
请求格式：

```
{
  "username":"邮箱",
  "password":"密码"
}
```

若操作成功，服务端应返回 HTTP 状态 `204 No Content`。

安全提示： 该 API 也可用于判断密码的正确性，因此应受到和登录 API 一样的速率限制。

会话部分



1. 服务端向待进服的玩家发送 serverId，其可以被视为一个随机的字符串。
2. 客户端将该 serverId 连同自己的 accessToken 发送给 Yggdrasil 端，请求验证。
3. 服务端将 serverId 及玩家的名字发送给 Yggdrasil 端，查询玩家是否已完成验证。
4. 若玩家验证成功，则 Yggdrasil 端将玩家的 Profile 返回给服务端，其中包含材质信息和数字签名。
5. 服务端将该玩家的 Profile 分发给服内所有玩家。

上图使用 draw.io 绘制，源文件见 [mc入服原理.drawio](#)。

该部分用于角色进入服务器时的验证。主要流程如下：

1. **Minecraft 服务端**和 **Minecraft 客户端**共同生成一段字符串（serverId），其可以被认为是随机的
2. **Minecraft 客户端**将 serverId 及令牌发送给 **Yggdrasil 服务端**（要求令牌有效）
3. **Minecraft 服务端**请求 **Yggdrasil 服务端**检查客户端会话的有效性，即客户端是否成功进行第 2 步

客户端进入服务器

POST /sessionserver/session/minecraft/join

记录服务端发送给客户端的 `serverId`，以备服务端检查。

请求格式：

```
{
  "accessToken": "令牌的 accessToken",
  "selectedProfile": "该令牌绑定的角色的 UUID（无符号）",
  "serverId": "服务端发送给客户端的 serverId"
}
```

仅当 `accessToken` 有效，且 `selectedProfile` 与令牌所绑定的角色一致时，操作才成功。

服务端应记录以下信息：

- `serverId`
- `accessToken`
- 发送该请求的客户端 IP

实现时请注意：以上信息应记录在内存数据库中（如 Redis），且应该设置过期时间（如 30 秒）。鉴于 `serverId` 的随机性，可以将其作为主键。

若操作成功，服务端应返回 HTTP 状态 `204 No Content`。

服务端验证客户端

```
GET /sessionserver/session/minecraft/hasJoined?username={username}&serverId={serverId}&ip={ip}
```

检查客户端会话的有效性，即数据库中是否存在该 `serverId` 的记录，且信息正确。

请求参数：

参数	值
username	角色的名称
serverId	服务端发送给客户端的 serverId
ip (可选)	Minecraft 服务端获取到的客户端 IP，仅当 <code>prevent-proxy-connections</code> 选项开启时包含

`username` 需要与 `serverId` 所对应令牌所绑定的角色的名称相同。

响应格式：

```
{
    // ... 令牌所绑定角色的完整信息（包含角色属性及数字签名，格式见 $角色信息的序列化）
}
```

若操作失败，服务端应返回 HTTP 状态 204 No Content。

角色部分

该部分用于角色信息的查询。

查询角色属性

```
GET /sessionserver/session/minecraft/profile/{uuid}?unsigned={unsigned}
```

查询指定角色的完整信息（包含角色属性）。

请求参数：

参数	值
uuid	角色的 UUID（无符号）
unsigned <i>（可选）</i>	true 或 false。是否在响应中 不包含 数字签名，默认为 true

响应格式：

```
{
    // ... 角色信息（包含角色属性。若 unsigned 为 false，还需要包含数字签名。格式见 $角色信息的序列化）
}
```

若角色不存在，服务端应返回 HTTP 状态 204 No Content。

按名称批量查询角色

```
POST /api/profiles/minecraft
```

批量查询角色名称所对应的角色。

请求格式：

```
[
  "角色名称"
  // ,... 还可以有更多
]
```

服务端查询各个角色名称所对应的角色信息，并将其包含在响应中。不存在的角色不需要包含。响应中角色信息的先后次序无要求。

响应格式：

```
[
  {
    // 角色信息（注意：不包含角色属性。格式见 §角色信息的序列化）
  }
  // ,...（可以有更多）
]
```

安全提示： 为防止 CC 攻击，需要为单次查询的角色数目设置最大值，该值至少为 2。

材质上传

```
PUT /api/user/profile/{uuid}/{textureType}
DELETE /api/user/profile/{uuid}/{textureType}
```

设置或清除指定角色的材质。

并非所有角色都可以上传皮肤和披风。要获取当前角色能够上传的材质类型，参见 [§ uploadableTextures](#) 可上传的材质类型。

请求参数：

参数	值
uuid	角色的 UUID（无符号）
textureType	材质类型，可以为 skin（皮肤）或 cape（披风）

请求需要带上 HTTP 头部 `Authorization: Bearer {accessToken}` 进行认证。若未包含 `Authorization` 头或 `accessToken` 无效，则返回 `401 Unauthorized`。

如果操作成功，则返回 `204 No Content`。

下面分别介绍 PUT 和 DELETE 这两个 HTTP 方法的用法：

PUT 上传材质

请求的 Content-Type 为 multipart/form-data ，请求载荷由以下部分组成：

名称 (name)	内容
model	(仅用于皮肤) 皮肤的材质模型，可以为 slim （细胳膊皮肤）或空字符串（普通皮肤）。
file	材质图像， Content-Type 须为 image/png 。 建议客户端设置 Content-Disposition 中的 filename 参数为材质图像的文件名，这可以被验证服务器用作材质的备注。

如果操作成功，则返回 204 No Content 。

DELETE 清除材质

清除材质后，该类型的材质将恢复为默认。

扩展 API

以下 API 是为了方便 authlib-injector 进行自动配置而设计的。

API 元数据获取

GET /

响应格式：

```
{
  "meta":{
    // 服务端的元数据，内容任意
  },
  "skinDomains":[ // 材质域名白名单
    "域名匹配规则 1"
    // ,...
  ],
  "signaturePublicKey":"用于验证数字签名的公钥"
}
```

`signaturePublicKey` 是 PEM 格式的公钥，用于验证角色属性的数字签名。其以 `-----BEGIN PUBLIC KEY-----` 开头，以 `-----END PUBLIC KEY-----` 结尾，中间允许出现换行符，但不允许出现其他空白字符（亦允许文末出现换行符）。

材质域名白名单

Minecraft 仅会从白名单中的域名下载材质。如果材质 URL 的域名不在白名单中，则会出现 `Textures payload has been tampered with (non-whitelisted domain)` 错误。采用此机制的原因见 [MC-78491](#)。

材质白名单默认包含 `.minecraft.net`、`.mojang.com` 两项规则，你可以设置 `skinDomains` 属性以添加额外的白名单规则。规则格式如下：

- 如果规则以 `.`（dot）开头，则匹配以这一规则结尾的域名。
 - 例如 `.example.com` 匹配 `a.example.com`、`b.a.example.com`，**不匹配** `example.com`。
- 如果规则**不以** `.`（dot）开头，则匹配的域名须与规则**完全相同**。
 - 例如 `example.com` 匹配 `example.com`，**不匹配** `a.example.com`、`eexample.com`。

meta 中的元数据

`meta` 中的内容没有强制要求，以下字段均为可选。

服务端基本信息

Key	Value
<code>serverName</code>	服务器名称
<code>implementationName</code>	服务端实现的名称
<code>implementationVersion</code>	服务端实现的版本

服务器网址

如果您需要在启动器中展示验证服务器首页地址、注册页面地址等信息，您可以在 `meta` 中添加一个 `links` 字段。

`links` 字段的类型是对象，其中可以包含：

Key	Value
<code>homepage</code>	验证服务器首页地址
<code>register</code>	注册页面地址

以下带有 **(advanced)** 标注的字段为高级选项，通常情况下**不需要**设置。

Key	Value
feature.non_email_login	布尔值，指示验证服务器是否支持使用邮箱之外的凭证登录（如角色名登录），默认为 false。 详情见 §使用角色名称登录 。
feature.legacy_skin_api	(advanced) 布尔值，指示验证服务器是否支持旧式皮肤 API，即 GET <code>/skins/MinecraftSkins/{username}.png</code> 。 当未指定或值为 false 时，authlib-injector 会使用内建的 HTTP 服务器在本地处理对该 API 的请求；若值为 true，请求将由验证服务器处理。 详情见 README § 参数 中的 - <code>Dauthlibinjector.legacySkinPolyfill</code> 选项。
feature.no_mojang_namespace	(advanced) 布尔值，是否禁用 authlib-injector 的 Mojang 命名空间（@mojang 后缀）功能，默认为 false。 详情见 README § 参数 中的 - <code>Dauthlibinjector.mojangNamespace</code> 选项。
feature.enable_mojang_anti_features	(advanced) 布尔值，是否开启 Minecraft 的 anti-features，默认为 false。 详情见 README § 参数 中的 - <code>Dauthlibinjector.mojangAntiFeatures</code> 选项。
feature.enable_profile_key	(advanced) 布尔值，指示验证服务器是否支持 Minecraft 的消息签名密钥对功能（即多人游戏中聊天消息的数字签名），默认为 false。 开启后，Minecraft 将通过 POST <code>/minecraftservices/player/certificates</code> 这一 API 获取验证服务器颁发的密钥对。当未指定或为 false 时，Minecraft 将不会在聊天消息中包含数字签名。 详情见 README § 参数 中的 - <code>Dauthlibinjector.profileKey</code> 选项。
feature.username_check	(advanced) 布尔值，指示 authlib-injector 是否启用用户名验证功能，默认为 false。 开启后用户名中带有特殊字符的玩家将无法加入

Key	Value
	服务器，详情见 README § 参数 中的 <code>-Dauthlibinjector.usernameCheck</code> 选项。

响应示例

```
{
  "meta": {
    "implementationName": "yggdrasil-mock-server",
    "implementationVersion": "0.0.1",
    "serverName": "yushijinhun's Example Authentication Server",
    "links": {
      "homepage": "https://skin.example.com/",
      "register": "https://skin.example.com/register"
    },
    "feature.non_email_login": true
  },
  "skinDomains": [
    "example.com",
    ".example.com"
  ],
  "signaturePublickey": "-----BEGIN PUBLIC KEY-----\nMIICIj... (省略) ...EAAQ==\n-----END PUBLIC KEY-----\n"
}
```

API 地址指示 (ALI)

API 地址指示 (API Location Indication, 简称 ALI) 是一个 HTTP 响应头字段 `X-Authlib-Injector-API-Location`，起到服务发现的作用。ALI 的值为相对 URL 或绝对 URL，它指向与当前页面相关联的 Yggdrasil API。

使用 ALI 后，用户只需输入一个与 Yggdrasil API 相关联的地址即可，不必输入真正的 API 地址。例如，`https://skin.example.com/api/yggdrasil/` 可以被简化为 `skin.example.com`。支持 ALI 的启动器会请求 `(https://)skin.example.com`，识别响应中的 ALI 头字段，并根据它找到真正的 API 地址。

皮肤站可以在首页，或在全站启用 ALI。启用 ALI 的方法为在 HTTP 响应中添加 `X-Authlib-Injector-API-Location` 头字段，例如：

```
X-Authlib-Injector-API-Location: /api/yggdrasil/ # 使用相对 URL
X-Authlib-Injector-API-Location: https://skin.example.com/api/yggdrasil/ # 亦可使用绝对 URL，支持跨域
```

当一个页面的 ALI 指向其本身时，这个 ALI 会被忽略。

参见

- [Authentication - wiki.vg](#)
- [Mojang API - wiki.vg](#)
- [Protocol - wiki.vg](#)

参考实现

[yggdrasil-mock](#) 为本规范的参考实现。